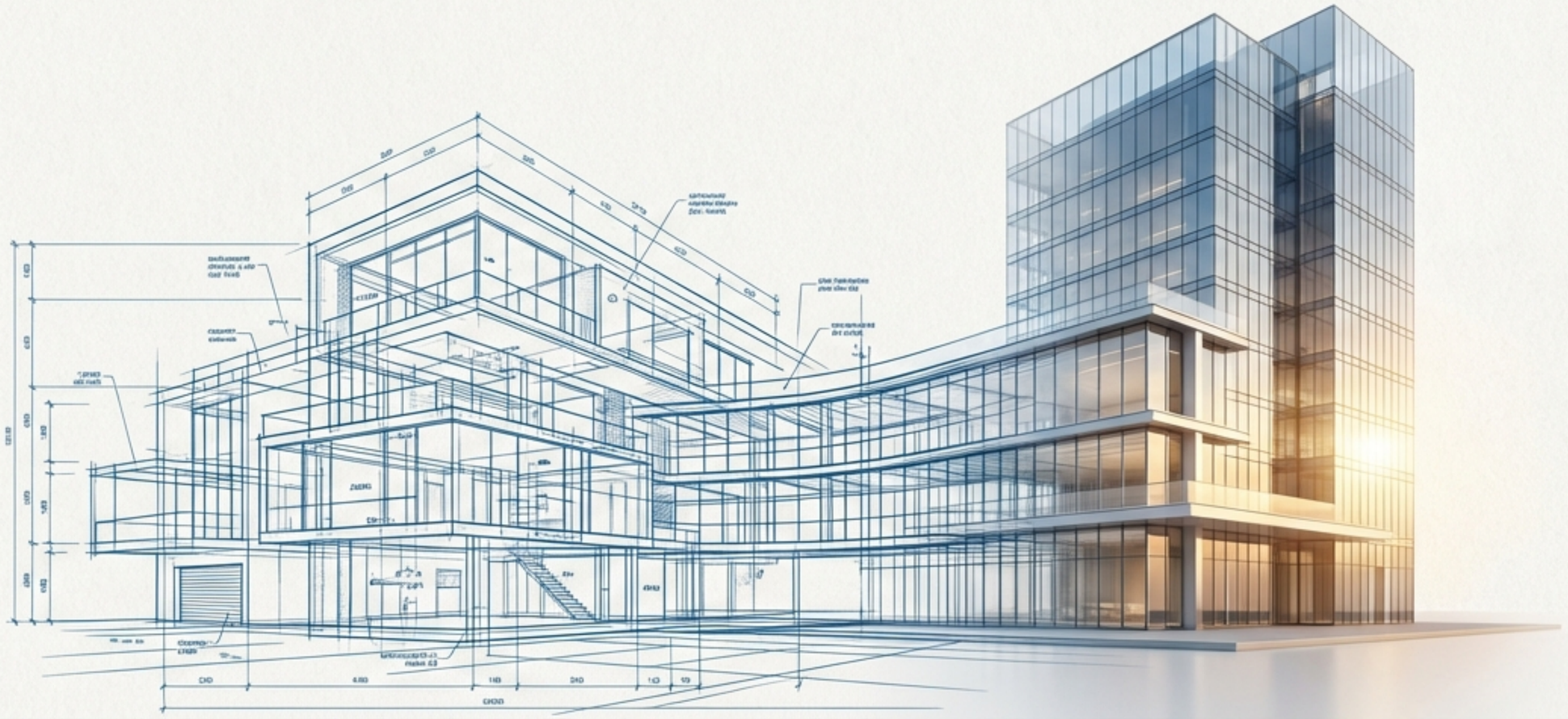
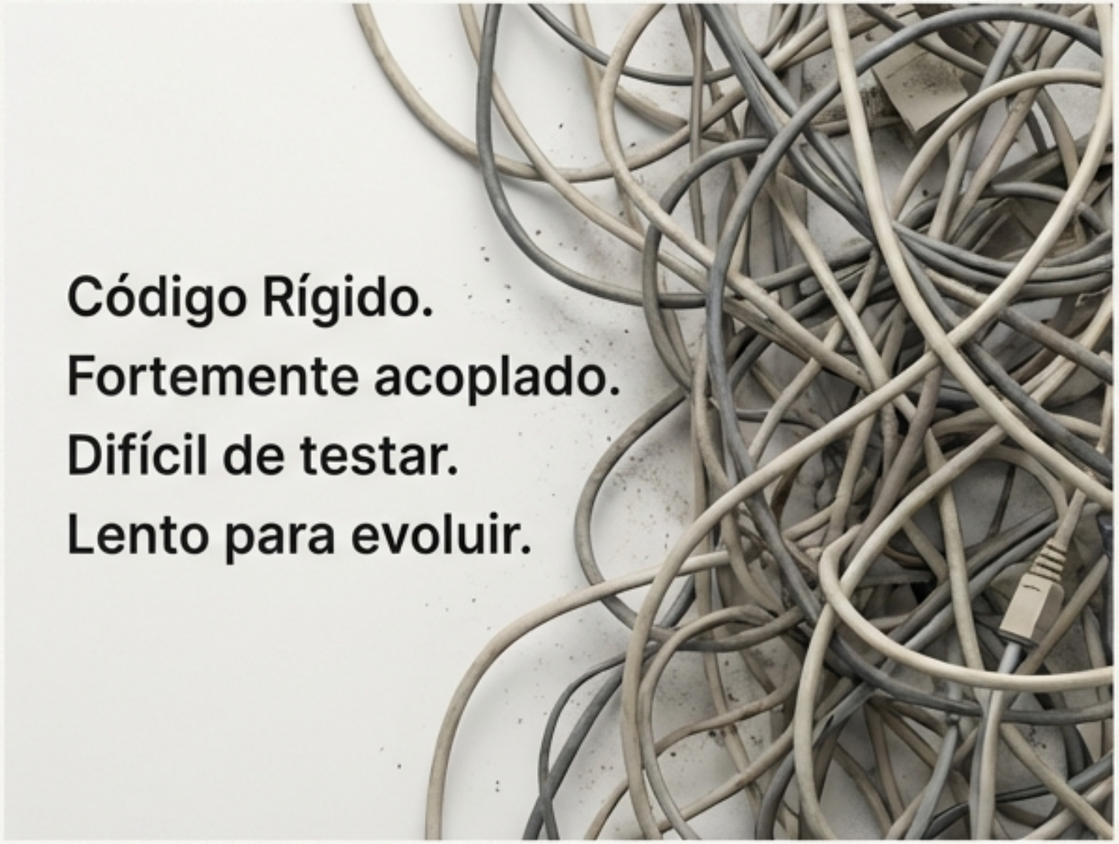
De Construtor a Arquiteto: O Salto para o ABAP Moderno

Dominando os pilares do código flexível e resiliente para S/4HANA, RAP e Clean Core.



NotebookLM

O Código que Você Escreve Hoje Define a Manutenção de Amanhã



Código Rígido.
Fortemente acoplado.
Difícil de testar.
Lento para evoluir.



Código Flexível.
Desacoplado.
Testável por design.
Rápido para adaptar.

Esta não é apenas uma aula sobre novos comandos. É uma mudança de mentalidade, a base para se tornar um **arquiteto de software** ABAP.

NotebookLM

Os 4 Pilares do Design de Software Sênior



1. O Contrato (Interfaces)

Como definir acordos claros que permitem a substituição de peças sem quebrar o sistema.



2. A Criação Centralizada (Factory)

Como isolar a lógica de instanciação de objetos para facilitar a manutenção futura.



3. A Resiliência (Exceções)

Como construir um tratamento de erros robusto e informativo, abandonando o `sy-subrc`.



4. A Estrutura Flexível (Composição)

Como escolher a ferramenta certa entre Herança e Interfaces para máxima flexibilidade.

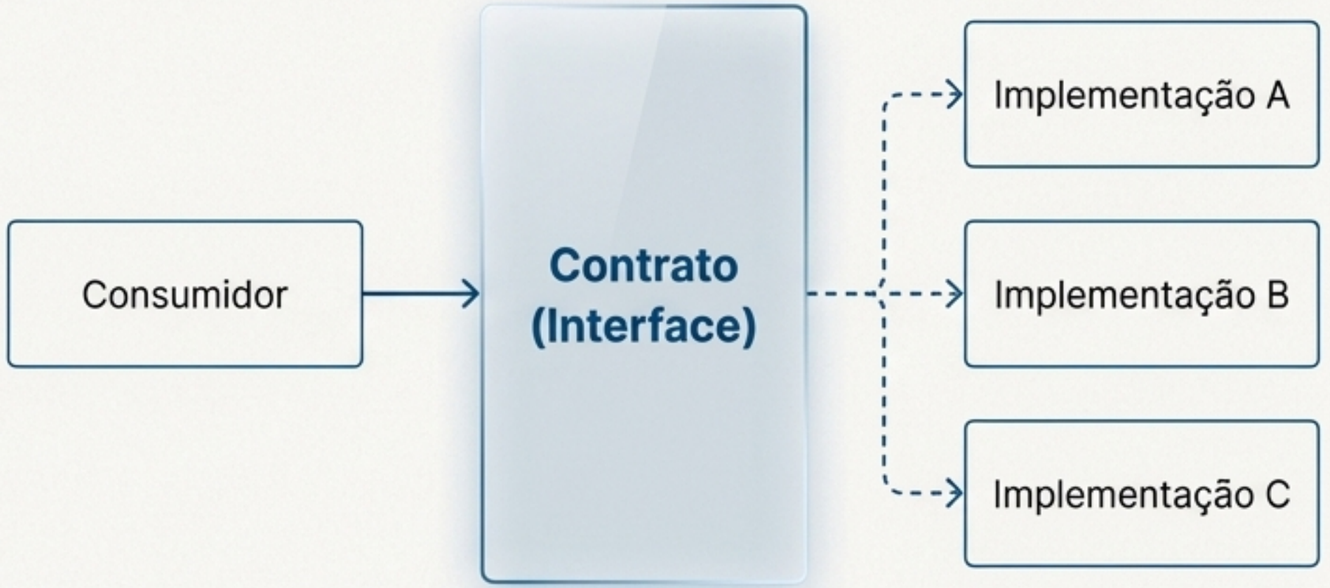
NotebookLM

Pilar 1: Interfaces - O Contrato do Código

"Em arquiteturas de software maduras, programar voltado para interfaces, e não para implementações, é um sinal de senioridade."

O que é uma Interface?

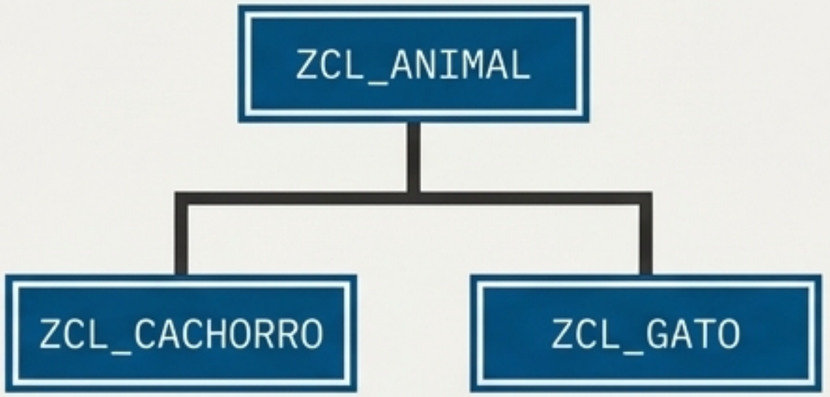
- É um contrato que define *o quê* uma classe deve fazer, mas não *como*.
- Contém apenas assinaturas de métodos, sem código de implementação.
- Garante que **diferentes classes "falem o mesmo idioma"**.



NotebookLM

Herança vs. Interface: A Batalha da Flexibilidade

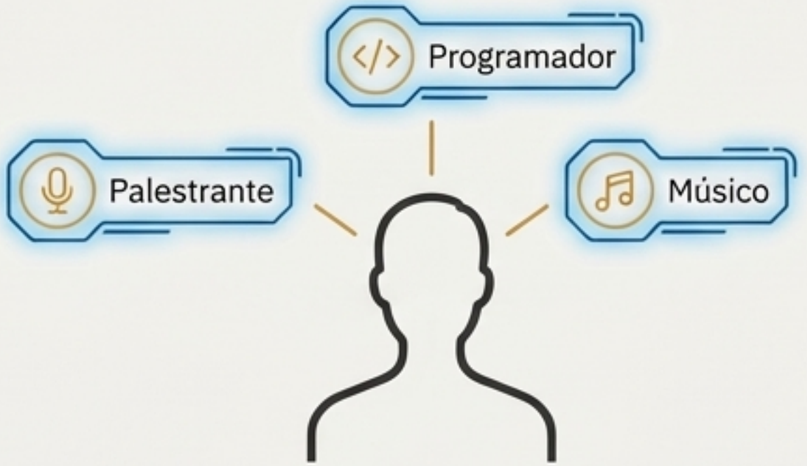
Herança (É um)



Um relacionamento estático.
"Um `ZCL_CACHORRO` **é um** `ZCL_ANIMAL`".

- Acoplamento forte.
- ABAP suporta apenas herança simples.
- Mudanças na superclasse podem quebrar as subclasses (“Fragile Base Class Problem”).

Interface (Faz algo)



Um relacionamento comportamental.
"Um `ZCL_CACHORRO` **faz** latido (ZIF_LATIDOR`)".

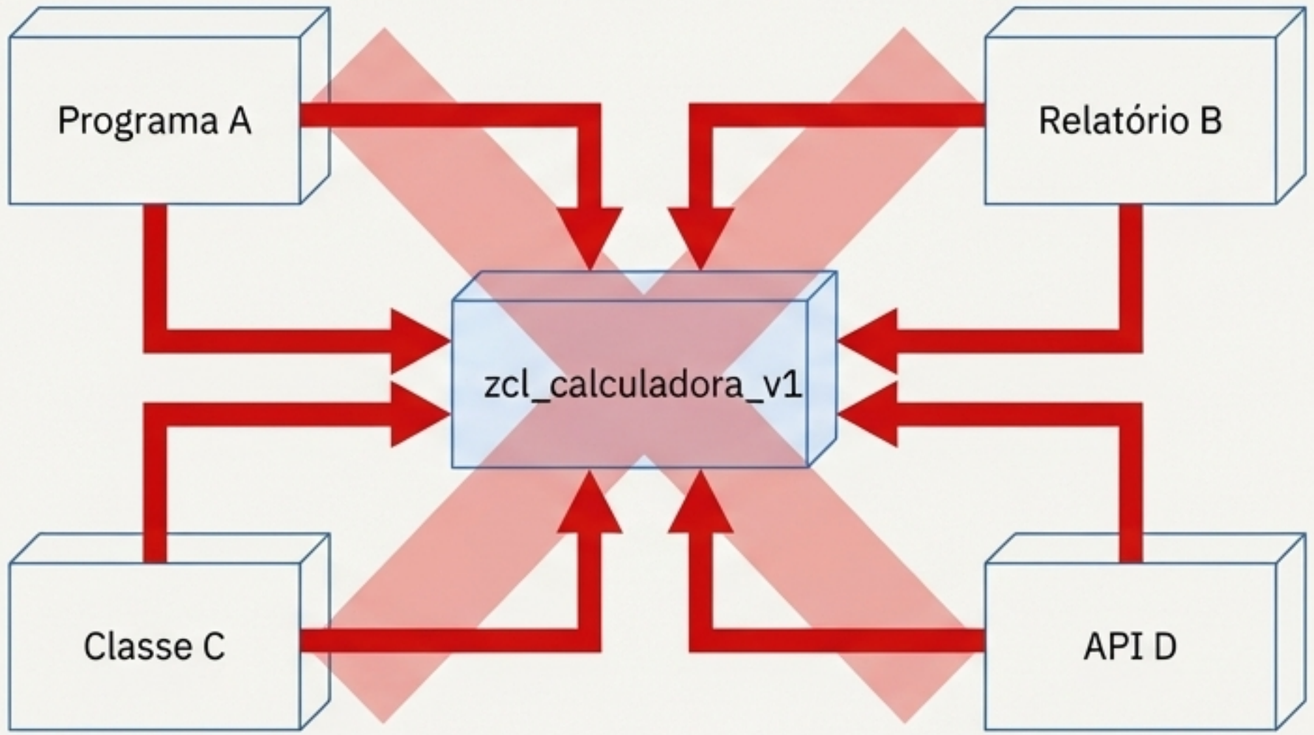
- Acoplamento fraco.
- Uma classe pode implementar ‘N’ interfaces, desempenhando múltiplos papéis.
- Essencial para **Polimorfismo** e **Testabilidade**.

NotebookLM

Pilar 2: Factory - A Criação Centralizada de Objetos

O Problema: O Acoplamento Forte da Instanciação Direta

O código `DATA(obj) = NEW zcl_calculadora_v1( )` espalhado por toda a aplicação 'casa' seu programa com uma implementação específica. Mudar para a `zcl_calculadora_v2` se torna um pesadelo de manutenção.



NotebookLM

O Padrão Factory em Ação: O Chef Decide a Receita

Conceito

O seu código (o cliente) pede um 'prato de massa' (a interface `zif_travel_calc`). A Factory (o chef) decide a receita (`zcl_travel_calc_br` ou `_us`) com base nos ingredientes (o país). O cliente não precisa conhecer a receita.



Benefício Chave: Para mudar a classe de cálculo do Brasil, você altera o código em **UM ÚNICO LUGAR**.

Código

```
CLASS-METHODS: get_calculator
    IMPORTING iv_country TYPE land1
    RETURNING VALUE(ro_calc) TYPE REF TO zif_travel_calc.

  METHOD get_calculator.
    CASE iv_country.
      WHEN 'BR'.
        ro_calc = NEW zcl_travel_calc_br( ).
      WHEN 'US'.
        ro_calc = NEW zcl_travel_calc_us( ).
      ...
    ENDCASE.
  ENDMETHOD.
```

Retorna a INTERFACE!

Lógica de decisão CENTRALIZADA

NotebookLM

Pilar 3: Exceções - A Resiliência do Código

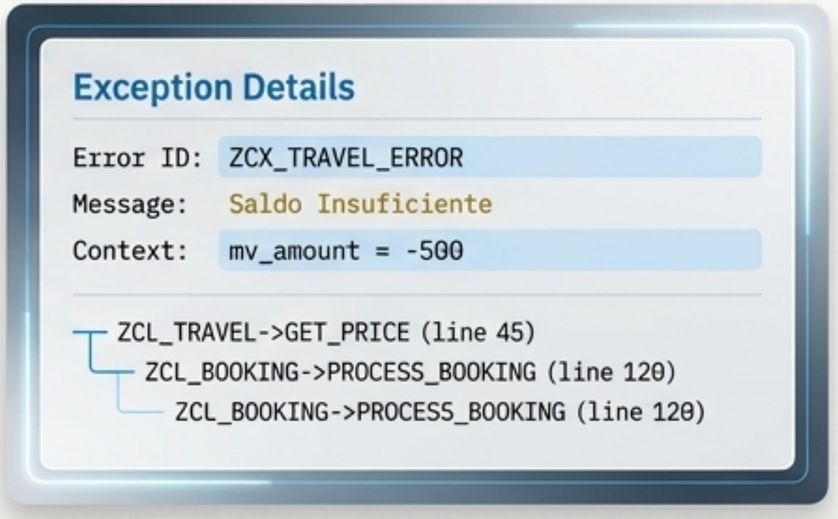
O Modelo Antigo: `EXCEPTIONS error = 1` + `CHECK sy-subrc`

O erro é mudo. O número `1` não informa o *porquê*, não carrega contexto (variáveis, textos) e não indica a origem exata do problema.



O Modelo Moderno: `RAISE EXCEPTION TYPE cx_...` + `TRY...CATCH`

O erro é um objeto rico em informação. Ele carrega mensagens T100, valores de variáveis, a pilha de chamadas e pode ser tratado de forma estruturada.



NotebookLM

A Hierarquia de Exceções: Escolhendo a Ferramenta Certa

CX_STATIC_CHECK (Checada)



Quando usar:

- Erros de negócio previsíveis onde o consumidor **deve** reagir.

Exemplos:

- ‘Saldo Insuficiente’, ‘Cliente Bloqueado’.

Comportamento:
O compilador **obriga** o tratamento com `TRY...CATCH` ou a propagação com `RAISING`.

CX_NO_CHECK (Não Checada)



Quando usar:

- Erros técnicos graves ou falhas de programação onde a recuperação é improvável.

Exemplos:

- ‘Divisão por Zero’, ‘Parâmetro Obrigatório Vazio’.

Comportamento:
O tratamento é **opcional**. Se não tratada, gera um **DUMP** informativo.

NotebookLM

O Desafio: Refatorando um Cálculo de Desconto Rígido

Cenário

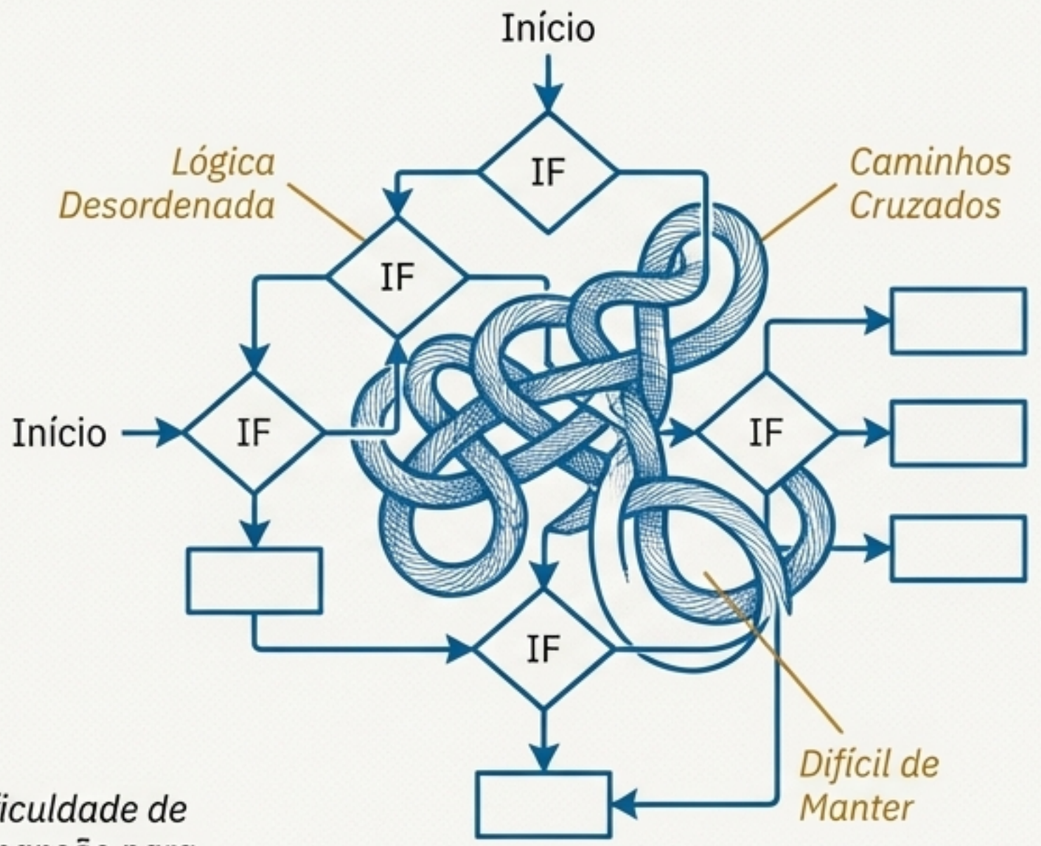
Precisamos calcular o desconto de uma viagem.

- **Clientes VIP** recebem 20% de desconto.
- **Clientes Standard** não recebem desconto.
- ****Requisito Futuro****: Novos tipos de clientes (ex: “Partner") devem ser fáceis de adicionar.

O Código "Antes" (O Problema)

```
// Lógica espalhada e difícil de estender
IF iv_customer_type = 'VIP'.
  lv_discount = iv_price * '0.2'.
ELSEIF iv_customer_type = 'STD'.
  lv_discount = 0.
ELSE.
  " O que acontece quando o tipo 'PARTNER' chegar?
  " Mais um ELSEIF...
ENDIF.
```

Dificuldade de expansão para novos tipos.

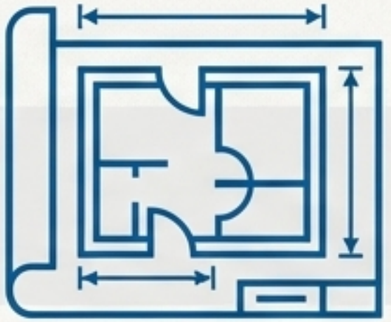


NotebookLM

A Solução - Passo 1: O Contrato e as Estratégias (Padrão Strategy)

Parte 1: A Interface (O Contrato)

Definimos *o que* precisa ser feito, ignorando *como*.



```
INTERFACE zif_discount_strategy PUBLIC.
  METHODS calculate ...
ENDINTERFACE.
```

Parte 2: As Implementações (As Estratégias)

Criamos classes pequenas e com responsabilidade única.

```
CLASS zcl_discount_vip IMPLEMENTS zif_discount_strategy.
  " Logic for 20% discount
ENDCLASS.
```



VIP Logic

```
CLASS zcl_discount_std IMPLEMENTS zif_discount_strategy.
  " Logic for 0% discount
ENDCLASS.
```



STD Logic

Princípio Aplicado: Separação de Preocupações. A classe VIP só sabe calcular o desconto VIP. Nada mais.

NotebookLM

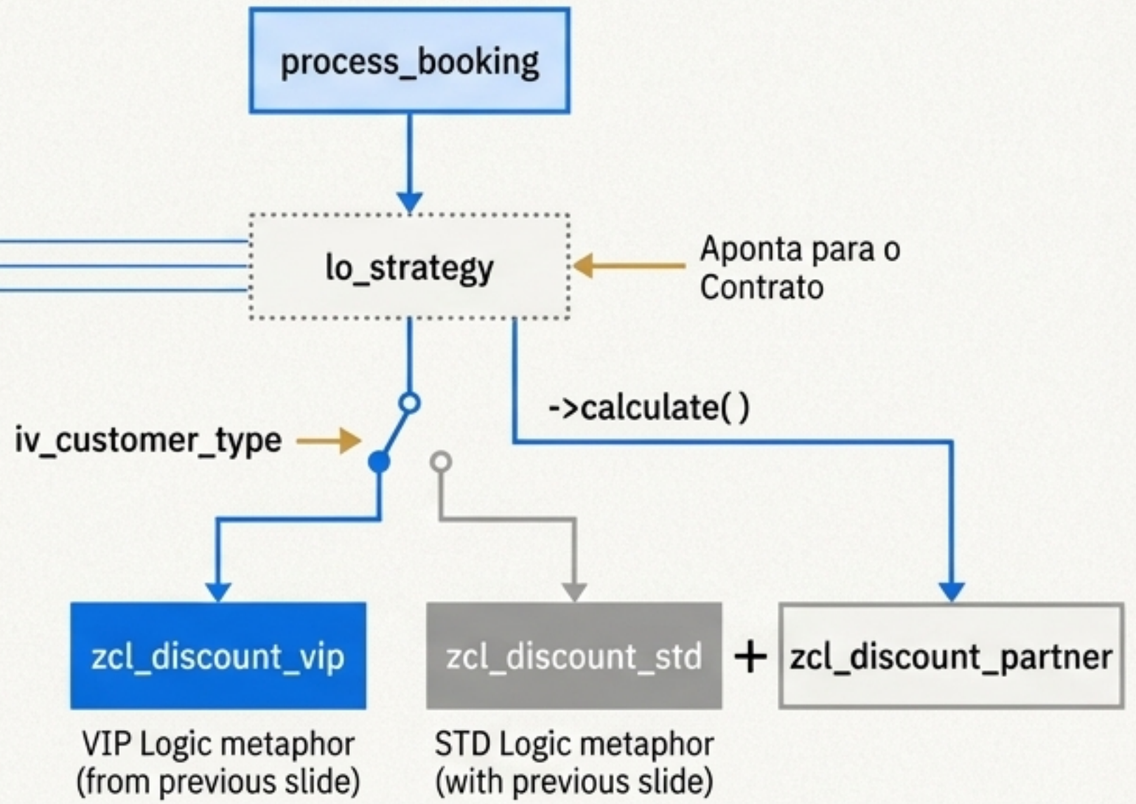
A Solução - Passo 2:
A Orquestração Limpa e Polimórfica

```
METHOD process_booking.

  // 1. Variável declarada com o tipo da INTERFACE
  DATA: lo_strategy TYPE REF TO zif_discount_strategy.

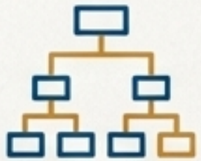
  // 2. Lógica de Criação (Factory Simples)
  IF iv_customer_type = 'VIP'.
    lo_strategy = NEW zcl_discount_vip( ).
  ELSE.
    lo_strategy = NEW zcl_discount_std( ).
  ENDIF.

  // 3. Execução Polimórfica
  DATA(lv_discount) = lo_strategy->calculate( 1000 ).
ENDMETHOD.
```



NotebookLM

Seu Arsenal de Arquiteto: Tabela de Decisão

Cenário	 Usar Herança (INHERITING FROM)	 Usar Interface (INTERFACES)
Relacionamento Conceitual	“É um” (Carro é um Veículo)	“Faz algo” / “Possui comportamento” (Carro é Dirigível)
Objetivo Principal	Reutilizar código existente da classe mãe para evitar duplicação.	Definir um contrato comum para classes distintas conversarem.
Acoplamento	**Alto/Forte**. Subclasses quebram se a mãe mudar.	**Baixo/Fraco**. A implementação pode mudar livremente desde que respeite o contrato.
Multiplicidade	**Não**. ABAP só permite 1 superclasse.	**Sim**. Uma classe pode implementar N interfaces.

NotebookLM

Seu Arsenal de Arquiteto: Glossário Essencial

Interface (ZIF)
Um contrato que define assinaturas de métodos sem implementação, permitindo o desacoplamento entre consumidor e provedor.

Polimorfismo
A capacidade de objetos de classes diferentes responderem à mesma chamada de método (definida na Interface) de maneiras específicas.

Factory Pattern
Padrão de criação que centraliza a decisão de qual classe concreta instanciar, escondendo essa lógica do código consumidor.

Exception Class (CX)
Objetos que transportam o contexto completo de um erro (texto, atributos, pilha) do ponto de falha até o ponto de tratamento (CATCH).

Separação de Preocupações
Princípio de design onde cada classe deve ter uma responsabilidade única e bem definida.

NotebookLM

O Próximo Nível é uma Questão de Design

Escrever código que *funciona agora* é a tarefa de um construtor. Projetar sistemas que *evoluem amanhã* é a marca de um arquiteto. Os princípios de desacoplamento, contratos claros e resiliência não são opcionais no mundo de S/4HANA e Clean Core – são a base do desenvolvimento sustentável.



NotebookLM